

Minor Project Synopsis Report

CampusGenie AI

**A Retrieval-Augmented, Agentic AI Framework for Conversational Access to
Institutional Academic Knowledge**

Project Category: University Problem Statement

Submitted in partial fulfilment of the requirement of the degree of

Masters of Communication Applications

(Section - A)

to

K.R Mangalam University

by

Team CYC (Control Your Career)

Team ID: 26O4043

Under the supervision of

Mr. Vishwanil Suman

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram - 122001, India

August 2026

Team Members: Team CYC — Control Your Career

Role	Name	ID
Team Leader	Bhanu Asati	2501560014
Member	Srashti Dwivedi	2501560012

Internal Mentor

Mr. Vishwanil Suman, Department of Computer Science and Engineering, School of Engineering and Technology, K.R Mangalam University.

INDEX

S.No.	Title
1.	Abstract
2.	Introduction
3.	Motivation
4.	Literature Review
5.	Research Gap Analysis
6.	Problem Statement
7.	Research Objectives
8.	Proposed System Design and Tools / Platforms Used
9.	Methodology
10.	Evaluation Methodology and Expected Outcomes
11.	Limitations and Future Scope
12.	References

ABSTRACT

Higher-education institutions accumulate large, heterogeneous, and continuously evolving repositories of information — timetables, examination notices, attendance policies, placement circulars, fee schedules, and administrative regulations — distributed across static portals, PDF circulars, spreadsheets, and departmental notice boards. Locating a specific, current fact within this landscape typically requires a student or faculty member to manually search several disconnected systems, since conventional portals expose only keyword search or fixed navigation and cannot interpret natural-language intent, synthesize information across sources, or verify the currency of what they return.

This project presents CampusGenie AI, an AI-powered academic assistant that unifies three complementary technical paradigms — Large Language Models (LLMs) for natural-language understanding and generation, Retrieval-Augmented Generation (RAG) for grounding model outputs in verifiable institutional documents, and Agentic AI orchestration for multi-step reasoning and tool use — into a single conversational system. Rather than relying on an LLM's parametric knowledge alone, which is static and prone to hallucination on institution-specific facts (Huang et al., 2023), CampusGenie AI retrieves semantically relevant passages from a vector-indexed institutional knowledge base at query time and conditions generation on that retrieved context, following the retrieval-augmented paradigm formalized by Lewis et al. (2020) and subsequently surveyed and extended by Gao et al. (2023).

Architecturally, the system follows a six-layer design: a React.js presentation layer captures natural-language queries; a FastAPI application layer performs request validation and routing; a LangChain/LangGraph agent layer, implemented in the ReAct reasoning-and-acting pattern (Yao et al., 2023), decides whether retrieval or additional tool invocation is required and orchestrates multi-step query resolution; a retrieval layer computes dense-vector similarity search over a ChromaDB/FAISS index (Johnson et al., 2019) populated with embedded institutional documents; and a generation layer synthesizes the final, context-grounded response using an LLM served through the OpenAI or Gemini API. The knowledge base is built by chunking source documents, embedding each chunk with a sentence-level encoder, and indexing the resulting vectors for approximate nearest-neighbour retrieval at query time.

The system's effectiveness is intended to be assessed empirically, using retrieval-and-generation quality metrics adapted from the RAGAS framework (Es et al., 2024) — namely context precision, context recall, faithfulness, and answer relevance — alongside end-to-end latency and qualitative usability evaluation with representative student and faculty queries. This evaluation-first framing distinguishes CampusGenie AI from ad hoc academic chatbot implementations and positions it as a reproducible, extensible reference architecture for institutional conversational AI.

By combining grounded retrieval with agentic, multi-step reasoning within a modular and extensible architecture, CampusGenie AI aims to substantially reduce the manual effort required to access institutional information, decrease the volume of repetitive queries handled by administrative staff, and establish a scalable foundation for a broader suite of intelligent academic services — including voice interaction, automatic document summarization, and personalized academic and placement recommendations. This work contributes a concrete demonstration of how LLMs, RAG, and Agentic AI can be jointly engineered, deployed, and evaluated within the higher-education domain.

Keywords: Academic Assistant, Large Language Models, Retrieval-Augmented Generation, Agentic AI, LangChain, LangGraph, Vector Database, Conversational AI, Natural Language Processing, Information Retrieval

INTRODUCTION

Higher-education institutions are, by nature, information-intensive organizations. On any given day, a university generates and updates class schedules, attendance registers, examination notifications, placement circulars, hostel and transport notices, fee-payment deadlines, and policy documents governing eligibility, grading, and conduct. This information is essential to the day-to-day functioning of both students and faculty, yet it is rarely consolidated into a single, coherent, and queryable system. Instead, it is dispersed across learning-management systems, ERP portals, PDF circulars, spreadsheets, e-mail threads, and physical notice boards, each maintained by a different department with its own update cadence and interface conventions.

This fragmentation imposes a real and recurring cost. A student trying to determine whether they meet the minimum attendance threshold to sit for an examination may need to consult an attendance dashboard, cross-reference it against a separate policy document specifying the threshold, and confirm the examination date from a third source — three independent lookups to answer what is, from the student's perspective, a single question. Multiplied across a student body of several thousand and a continuous stream of similarly compound queries, the cumulative time cost — and the corresponding load placed on faculty and administrative staff who are asked to resolve the same questions manually — becomes substantial.

Conventional university information systems are not designed to absorb this cost. They are largely static: built around predefined navigation structures and keyword search, they cannot interpret the intent behind a naturally phrased question, cannot combine information drawn from more than one underlying source, and offer no mechanism for verifying that the information presented is current. These limitations are structural rather than incidental — they follow directly from the absence of any natural-language understanding or reasoning layer between the user and the underlying data.

Recent progress in Artificial Intelligence has substantially narrowed this gap. Large Language Models, built on the Transformer architecture (Vaswani et al., 2017), now demonstrate strong natural-language understanding and generation capability, making conversational interaction over unstructured institutional content technically feasible for the first time at scale. However, LLMs used in isolation are trained on static, general-purpose corpora and are consequently unable to answer questions about an institution's current notices, evolving policies, or private records; when pressed for such information, they are prone to hallucination — the confident generation of plausible but incorrect content (Huang et al., 2023). Retrieval-Augmented Generation (RAG) directly addresses this weakness by pairing the LLM with a retrieval mechanism that fetches relevant, up-to-date passages from an external knowledge base at inference time and conditions generation on that retrieved evidence (Lewis et al., 2020; Gao et al., 2023), so that responses remain grounded in the institution's own, verifiable documents rather than the model's frozen training data.

A further limitation of naive, single-pass RAG is that many realistic academic queries are compound — they require combining information from more than one source, or deciding, based on the query itself, whether retrieval is even necessary. Agentic AI addresses this by allowing a language model to reason over multiple steps, decide which tools or retrieval calls to invoke, and orchestrate a sequence of actions before producing a final answer, following patterns such as ReAct (Yao et al., 2023) and the broader class of LLM-based autonomous agents surveyed by Wang et al. (2023). Frameworks such as LangChain and LangGraph operationalize these patterns as reusable, composable agent workflows.

This project, CampusGenie AI, brings these three strands — LLMs, RAG, and Agentic AI — together into a single, coherent system purpose-built for the university context. The proposed assistant allows students and faculty to ask everyday academic questions in natural language and receive accurate, grounded, and conversational answers, with the underlying agent deciding when and how to retrieve institutional information rather than requiring the user to know where that information lives. In doing so, the project aims to demonstrate a rigorous, evaluable path from academic-information fragmentation to unified, intelligent conversational access, and to lay a scalable and extensible foundation for future AI-driven academic services.

MOTIVATION

The motivation for this project arises from a pattern observed repeatedly across university life: students spend a disproportionate amount of time not learning, but searching — for their attendance percentage, the date of an upcoming examination, a placement drive's eligibility criteria, or the correct hostel or fee-related procedure. This information is almost always present somewhere within the institution's systems, yet the effort required to locate and reconcile it is often comparable to the effort of answering the underlying academic question itself. This is an inefficiency of access, not of content, and it is precisely the kind of problem that a well-grounded conversational interface is suited to solve.

A closely related motivation concerns faculty and administrative workload. A significant share of the queries directed at faculty coordinators, class representatives, and administrative staff are repetitive and factual in nature — the same handful of questions asked by different students at different times. Each such query, however small, interrupts other work and does not scale with a growing student body. An assistant capable of correctly and consistently answering these repetitive queries would free faculty and staff time for higher-value interactions, without requiring any change to how the institution stores or manages its underlying data.

A third motivation is technical and forward-looking. LLMs, RAG, and Agentic AI have, within a short span, moved from research curiosities to production-grade techniques used to build reliable, grounded assistants across domains such as customer support, legal research, and enterprise knowledge management. Retrieval-augmented architectures in particular have been shown to reduce hallucination and improve factual grounding relative to LLMs used without retrieval (Lewis et al., 2020; Gao et al., 2023), while agentic frameworks extend this further by enabling multi-step reasoning over multiple information sources (Yao et al., 2023; Wang et al., 2023). Applying these techniques to the university domain is both timely and under-explored: most existing university chatbots predate this generation of RAG and agentic techniques and rely instead on rigid, rule-based or intent-classification approaches with narrow, hand-curated knowledge bases.

There is, therefore, a clear and current opportunity: to design, build, and rigorously evaluate a conversational academic assistant that (i) is grounded in an institution's actual, current documents rather than an LLM's static training data, (ii) can reason across multiple pieces of information to answer compound queries, and (iii) is architected to be extensible, so that capabilities such as voice interaction, document summarization, and personalized recommendations can be layered on without redesigning the system. CampusGenie AI is motivated by exactly this opportunity — to reduce the time students and faculty spend searching for information, to ease the repetitive query burden on administrative staff, and to produce a reference implementation whose design choices and evaluation methodology can inform future AI-driven academic systems, both at K.R Mangalam University and more broadly.

LITERATURE REVIEW

4.1 Foundations: Transformers and Large Language Models

The Transformer architecture, introduced by Vaswani et al. (2017), replaced recurrence with self-attention, enabling models to be trained at unprecedented scale and giving rise to the current generation of Large Language Models. Pretrained on vast, general-purpose text corpora, these models exhibit strong emergent capability in language understanding, few-shot reasoning, and fluent text generation, which is what makes a natural-language conversational interface to institutional data technically viable. However, an LLM's knowledge is fixed at the point of training: it has no visibility into an institution's current notices, evolving attendance records, or newly issued placement circulars, and it cannot be relied upon to answer such questions from parametric memory alone.

4.2 Hallucination and the Case for Grounding

Huang et al. (2023) provide a comprehensive taxonomy of hallucination in LLMs, distinguishing factuality hallucination (fabricated or incorrect facts) from faithfulness hallucination (outputs inconsistent with provided context or instructions), and survey the principal causes, spanning training-data noise, exposure bias, and the absence of an explicit mechanism to verify claims against a ground truth. For an institutional assistant, unmitigated hallucination is not a cosmetic weakness but a functional failure — an incorrect answer about an examination date or an eligibility threshold carries real consequences for the student who acts on it. This risk motivates the retrieval-grounded design adopted in this project, in which every generated answer is conditioned on retrieved, verifiable institutional text rather than the model's un-grounded parametric knowledge.

4.3 Dense Retrieval and Vector Search

Effective grounding depends on effective retrieval. Karpukhin et al. (2020) showed that dense vector representations, learned so that semantically related text is mapped to nearby points in an embedding space, substantially outperform traditional sparse, keyword-based retrieval (e.g., TF-IDF or BM25) for open-domain question answering, particularly when a query is phrased differently from the source text it should retrieve — a common occurrence in natural, conversational queries. Reimers and Gurevych (2019) proposed Sentence-BERT, a Siamese network architecture that produces sentence-level embeddings efficiently enough for large-scale similarity search, and which remains a standard building block of modern retrieval pipelines. At the systems level, Johnson, Douze, and Jégou (2019) introduced FAISS, an efficient library for approximate nearest-neighbour search over billions of vectors, enabling the sub-second retrieval latency that a conversational system requires.

4.4 Retrieval-Augmented Generation

Lewis et al. (2020) formalized Retrieval-Augmented Generation (RAG) as an architecture in which a differentiable retriever selects relevant passages from an external corpus and a sequence-to-sequence generator conditions its output on those passages, combining the parametric knowledge of the LLM with the non-parametric, updatable knowledge of the external corpus. This design allows a general-purpose LLM to be adapted to a specific, dynamic knowledge domain — such as a university's document repository — without any retraining of the underlying model, simply by updating the retrieval index. Gao et al. (2023), in a comprehensive survey, organize the subsequent RAG literature into naive RAG (a single retrieve-then-generate pass), advanced RAG (incorporating query rewriting, re-ranking, and iterative retrieval), and modular RAG (composable retrieval, routing, and generation

modules), and catalogue design choices — chunking strategy, embedding model selection, retrieval top-k, and re-ranking — that materially affect end-to-end answer quality. CampusGenie AI is positioned within this design space as a modular RAG system whose retrieval behaviour is further mediated by an agentic decision layer.

4.5 Agentic AI and Multi-Step Reasoning

A single retrieve-then-generate pass is insufficient for compound queries that require combining information from multiple sources or deciding, based on the query, whether retrieval is necessary at all. Yao et al. (2023) proposed ReAct, a prompting pattern in which a language model interleaves explicit reasoning traces ("thoughts") with concrete actions (such as tool or retrieval calls) and observations of their results, allowing the model to plan, adapt, and self-correct across multiple steps rather than committing to a single, unexamined response. Wang et al. (2023), in a broad survey of LLM-based autonomous agents, propose a unified framework spanning an agent's profiling, memory, planning, and action modules, situating ReAct-style reasoning as one instance of a much larger design space for agentic systems. LangChain and LangGraph operationalize these patterns as composable, production-oriented agent frameworks, providing the orchestration layer through which CampusGenie AI's agent decides when to retrieve, which tool to invoke, and how to assemble a final, multi-part answer.

4.6 Evaluating Retrieval-Augmented Systems

Because a RAG pipeline has multiple interacting components, its evaluation cannot rely on generation quality metrics alone. Es et al. (2024) introduced RAGAs, a reference-free evaluation framework built around four complementary metrics: context precision and context recall, which assess whether the retriever surfaces the passages actually needed to answer the query without excessive irrelevant content; faithfulness, which measures whether every claim in the generated answer is supported by the retrieved context; and answer relevance, which measures whether the generated answer actually addresses the question asked. This framework allows retrieval quality and generation quality to be diagnosed separately, which is essential for iterative system improvement, and is adopted in this project as the primary basis for evaluating CampusGenie AI (Section 10).

4.7 AI-Based Educational and Institutional Assistants

A parallel body of work has examined conversational AI within educational settings, including FAQ chatbots, academic-advising bots, and intent-classification-based student-support systems. These systems demonstrate the underlying premise that natural-language interfaces can meaningfully reduce the effort required to access institutional information. However, most rely on small, hand-curated intent sets and rule-based response templates rather than open, retrieval-grounded access to an institution's full document repository, and few incorporate the multi-step, agentic reasoning necessary to resolve compound queries — the specific combination of capabilities that CampusGenie AI is designed to provide.

4.8 Summary of Reviewed Literature

S.No.	Author(s)	Title	Source	Year	Key Findings / Relevance
1	Vaswani, A. et al.	Attention Is All You Need	NeurIPS	2017	Introduced the Transformer

					architecture and self-attention mechanism, the foundational neural network design underlying every modern LLM used in conversational AI systems.
2	Reimers, N., Gurevych, I.	Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks	EMNLP	2019	Proposed Siamese BERT networks for producing semantically meaningful sentence embeddings efficiently, foundational to modern dense-retrieval pipelines used in RAG.
3	Karpukhin, V. et al.	Dense Passage Retrieval for Open-Domain Question Answering	EMNLP	2020	Demonstrated that learned dense vector representations substantially outperform sparse (keyword-based) retrieval for open-domain question answering, underpinning modern vector-search retrieval.
4	Lewis, P. et al.	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	NeurIPS	2020	Introduced the RAG framework, combining a neural retriever with a generative model so that outputs are conditioned on retrieved external

					documents rather than parametric memory alone.
5	Johnson, J., Douze, M., Jégou, H.	Billion-Scale Similarity Search with GPUs	IEEE Trans. Big Data	2019	Described FAISS, a library for efficient large-scale approximate nearest-neighbour vector similarity search, widely used as the retrieval backbone of RAG pipelines.
6	Yao, S. et al.	ReAct: Synergizing Reasoning and Acting in Language Models	ICLR	2023	Proposed interleaving explicit reasoning traces with tool-use actions in LLMs, the reasoning pattern underlying most modern LLM agent frameworks, including LangGraph.
7	Wang, L. et al.	A Survey on Large Language Model based Autonomous Agents	arXiv:2308.11432	2023	Provided a unified framework surveying LLM-based agent construction, covering profiling, memory, planning, and action modules relevant to agentic system design.
8	Gao, Y. et al.	Retrieval-Augmented Generation for Large Language Models: A Survey	arXiv:2312.10997	2023	Systematized the RAG design space into naive, advanced, and modular paradigms and

					catalogued techniques for chunking, retrieval, and generation quality improvement.
9	Huang, L. et al.	A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions	arXiv:2311.05232	2023	Established a taxonomy of LLM hallucination and its causes, motivating retrieval-grounded generation as a principal mitigation strategy adopted in this project.
10	Es, S. et al.	RAGAs: Automated Evaluation of Retrieval Augmented Generation	EACL (System Demonstrations)	2024	Introduced reference-free metrics — faithfulness, answer relevance, and context relevance/precision/recall — for evaluating RAG pipelines, adopted as this project's evaluation framework.
11	Jurafsky, D., Martin, J. H.	Speech and Language Processing (3rd ed.)	Pearson Education	2023	Provided foundational NLP concepts — tokenization, embeddings, language modelling — underlying the retrieval and generation components of conversational AI systems.

RESEARCH GAP ANALYSIS

The literature reviewed in Section 4 points to considerable progress along three separate axes — large language models, retrieval-augmented generation, and agentic reasoning — but comparatively little work that integrates all three within a system purpose-built for, and evaluated against, the university information-access problem. This section analyses the specific gaps that CampusGenie AI is designed to address.

5.1 Fragmentation Between Fluency and Grounding

General-purpose LLM chat interfaces are fluent and capable of interpreting naturally phrased questions, but they are not grounded in any specific institution's current data; asked about a university's own policies or notices, they can only respond from static, general-purpose training data, or decline to answer. Conversely, existing rule-based or FAQ-style academic chatbots are grounded, in the narrow sense that their answers are drawn from a curated set of predefined responses, but they are rigid — a query phrased even slightly differently from the expected pattern, or one that combines two topics, typically falls outside their coverage. No single deployed system in the reviewed literature combines LLM-level fluency with document-level grounding over an institution's own, continuously updated knowledge base — precisely the gap RAG is designed to close (Lewis et al., 2020; Gao et al., 2023), but which remains rarely applied end-to-end in the university domain.

5.2 Absence of Multi-Step, Agentic Reasoning

Most existing RAG-based assistants, in and outside education, perform a single retrieve-then-generate pass per query. This is sufficient for simple factual lookups but insufficient for compound academic queries — for example, a query that requires checking a student's attendance record and then cross-referencing it against a separate eligibility policy document before producing a single, synthesized answer. Agentic frameworks such as ReAct (Yao et al., 2023) and the broader agent design space surveyed by Wang et al. (2023) exist to address exactly this class of problem, yet their application to institutional academic assistants is largely unexplored in the reviewed literature, leaving a gap between what agentic AI can do and what deployed academic chatbots actually do.

5.3 Lack of Rigorous, Component-Level Evaluation

A further, methodological gap concerns evaluation. Many existing academic chatbot projects are evaluated informally or not at all — success is asserted rather than measured. The RAG literature has, in contrast, converged on component-level, reference-free evaluation metrics such as those in the RAGAs framework (Es et al., 2024), which separately quantify retrieval quality (context precision and recall) and generation quality (faithfulness and answer relevance). Applying this evaluation discipline to an institutional assistant — rather than relying on anecdotal demonstration — is uncommon in the reviewed academic-chatbot literature and represents a gap this project directly addresses (Section 10).

5.4 Limited Architectural Extensibility

Finally, few existing systems are designed, from the outset, as modular platforms that can absorb new capabilities — voice interaction, document summarization, personalized recommendation — without a substantial redesign. Systems built around a fixed set of hand-written intents are structurally difficult to extend, since every new capability requires new, manually authored rules; a layered architecture

that separates presentation, orchestration, retrieval, and generation, as adopted here, is comparatively straightforward to extend by adding new tools to the agent layer.

5.5 Summary of the Identified Gap

Dimension	General-Purpose LLM Chat	Rule-Based / FAQ Chatbot	CampusGenie AI (RAG + Agentic AI)
Grounding in institution-specific data	Absent — general LLM chat has no access to the institution's own documents	Present but static — fixed intents/FAQ entries curated manually	Present, dynamic — RAG over a continuously updatable vector index
Handling naturally phrased, varied queries	Strong — LLM fluency handles paraphrase well	Weak — limited to predefined intents/patterns	Strong — LLM fluency combined with semantic retrieval
Multi-step / compound query resolution	Absent — single-turn response only	Absent — one intent maps to one canned answer	Present — agentic (ReAct-style) multi-step reasoning
Factual reliability on current information	Low — prone to hallucination on unseen facts	High, but only within the curated intent set	High — answers grounded in retrieved, current documents
Extensibility to new capabilities	Limited — bound by the base model's abilities	Limited — every new capability needs new hand-written intents	High — modular architecture supports adding tools/agents

CampusGenie AI is designed to directly close this gap by combining a RAG retrieval engine grounded in institutional documents with a LangChain/LangGraph agent layer capable of multi-step reasoning, evaluated with component-level RAG metrics, within a modular architecture explicitly designed for future extension.

PROBLEM STATEMENT

Universities generate and continuously update a large, heterogeneous volume of information — timetables, examination notices, attendance records, placement circulars, and policy documents — that students and faculty must locate and interpret across multiple, disconnected systems. Conventional university portals are static and non-conversational: they expose fixed navigation structures and, at best, keyword search, and are structurally unable to interpret natural-language intent, synthesize information drawn from more than one source, or verify that the information they return is current.

More precisely, the problem this project addresses can be stated as follows: given (i) a heterogeneous, continuously updated corpus of institutional documents, and (ii) a naturally phrased student or faculty query — which may require locating a single fact, synthesizing information from multiple documents, or reasoning over the retrieved information before responding — design a system capable of returning an accurate, grounded, natural-language answer, with the retrieval and reasoning process remaining transparent and auditable, and without requiring any change to how the institution already stores its underlying data.

Existing approaches address only part of this problem. General-purpose LLM assistants are fluent but ungrounded, and are consequently unreliable for institution-specific facts, since their knowledge is fixed at training time and cannot reflect a university's current notices or policies (Section 4.1–4.2). Rule-based or FAQ-style academic chatbots are grounded, but only within a narrow, manually curated set of predefined intents, and cannot handle naturally varied phrasing or compound, multi-source queries. Neither class of system incorporates the multi-step reasoning needed to resolve queries that require combining more than one piece of institutional information, nor is either typically subjected to rigorous, component-level evaluation of retrieval and generation quality.

There is, therefore, a well-defined and currently unmet need for an intelligent, retrieval-grounded, and agentic conversational assistant — one that combines the natural-language fluency and reasoning capability of Large Language Models with retrieval grounded in an institution's own documents, and the ability to orchestrate multi-step reasoning where a single query spans more than one information source. The proposed CampusGenie AI system is designed to meet this need, integrating LLMs, RAG, and Agentic AI within a single, evaluable architecture, and to do so in a way that is measurably more accurate, more flexible, and more extensible than either general-purpose LLM chat or conventional rule-based academic chatbots.

RESEARCH OBJECTIVES

The overarching objective of this project is to design, implement, and empirically evaluate CampusGenie AI, an AI-powered academic assistant that provides accurate, grounded, and conversational access to institutional knowledge. This overarching objective is decomposed into the following specific, evaluable objectives.

Primary Objectives

- O1 — Conversational Interface: Develop a natural-language conversational front end, built with React.js, through which students and faculty can pose academic queries in plain language rather than navigating static menus or performing keyword search.
- O2 — Grounded Retrieval: Design and implement a Retrieval-Augmented Generation pipeline — document chunking, embedding, vector indexing (ChromaDB/FAISS), and top-k semantic retrieval — that grounds every generated answer in the institution's own, current documents, directly mitigating the hallucination risk documented by Huang et al. (2023).
- O3 — Agentic Reasoning: Implement a LangChain/LangGraph agent layer, following the ReAct reasoning-and-acting pattern (Yao et al., 2023), capable of deciding when retrieval is required and orchestrating multi-step reasoning for compound queries that span more than one institutional document or data source.
- O4 — Rigorous, Component-Level Evaluation: Evaluate the resulting system using retrieval and generation quality metrics adapted from the RAGAs framework (Es et al., 2024) — context precision, context recall, faithfulness, and answer relevance — together with end-to-end response latency, rather than relying on anecdotal demonstration.

Secondary Objectives

- O5 — Modular, Extensible Architecture: Design the system as a layered, modular architecture (presentation, application, agent, retrieval, data, and generation layers) so that future capabilities — voice interaction, document summarization, assignment assistance, personalized recommendations — can be added as new agent tools without a structural redesign.
- O6 — Usable, Low-Barrier Interface: Provide a simple, intuitive web interface usable by students and faculty without any technical training, minimizing the barrier to adoption.
- O7 — Applied Research Skill-Building: Gain rigorous, hands-on experience in AI application development, prompt and retrieval engineering, conversational-interface design, and empirical evaluation methodology for LLM-based systems, of direct relevance to further research or industry work in applied AI.

PROPOSED SYSTEM DESIGN AND TOOLS / PLATFORMS USED

The development of CampusGenie AI requires tools that jointly support Large Language Model orchestration, Retrieval-Augmented Generation, Agentic AI workflows, and production-grade web application development. Each choice below is justified against the alternatives it was weighed against, rather than treated as a default.

Frontend — React.js

React.js was selected for its component-based architecture, which maps naturally onto a chat-style interface (message list, input composer, citation/source display) and supports incremental rendering of streamed model output — an important usability property for a conversational assistant, where perceived latency matters as much as total response time.

Backend — Python with FastAPI

FastAPI was chosen over alternatives such as Flask or Django for its native asynchronous request handling, which is well suited to I/O-bound workloads such as calling an external LLM API and querying a vector database concurrently, and for its automatic OpenAPI/Swagger documentation, which accelerates integration between the frontend and backend teams. Python's mature AI/NLP ecosystem — LangChain, LangGraph, embedding libraries, and vector-database clients — makes it the natural implementation language for the retrieval and agent layers.

AI Orchestration — LangChain and LangGraph

LangChain provides the building blocks for connecting an LLM, a retriever, and external tools into a single callable chain, while LangGraph extends this into explicit, stateful, graph-structured agent workflows — a closer fit to the ReAct reasoning-and-acting pattern (Yao et al., 2023) than a purely linear chain, since it allows the agent to branch, loop back to retrieval, or terminate based on intermediate reasoning state.

LLM Providers — OpenAI API and Gemini API

Supporting both the OpenAI and Gemini APIs, rather than committing to a single provider, allows the system to be evaluated across models with different cost, latency, and context-window characteristics, and provides a fallback path should one provider's API experience degraded availability.

Retrieval Paradigm and Embedding Model

The system adopts the Retrieval-Augmented Generation paradigm (Lewis et al., 2020) rather than fine-tuning an LLM directly on institutional data, since RAG allows the knowledge base to be updated simply by re-indexing new or changed documents, without the cost, latency, and staleness risk of periodic model retraining. Document chunks and incoming queries are mapped into a shared embedding space using a sentence-level dense encoder in the tradition of Sentence-BERT (Reimers & Gurevych, 2019), enabling semantic — rather than purely lexical — matching between a naturally phrased query and the institutional text that answers it, consistent with the dense-retrieval findings of Karpukhin et al. (2020).

Vector Database — ChromaDB and FAISS

ChromaDB is used during development for its simple, Python-native API and ease of local iteration, while FAISS (Johnson et al., 2019) is evaluated as a production-oriented alternative for its highly optimized approximate nearest-neighbour search at scale; both are compatible with the same embedding pipeline, allowing the choice to be revisited based on measured latency and index-build time as the knowledge base grows.

Evaluation Framework

Rather than adopting evaluation as an afterthought, the system's retrieval and generation quality are measured using metrics adapted from the RAGAs framework (Es et al., 2024) — see Section 10 — so that architectural and prompting decisions can be validated empirically rather than by inspection alone.

Version Control — Git and GitHub

Git and GitHub are used for source-code version control, branch-based feature development, and collaborative review across the team, and provide the audit trail necessary for a reproducible academic project.

Category	Tool / Technology	Purpose / Rationale
Frontend	React.js	Component-based, responsive conversational interface for submitting queries and rendering grounded, citation-aware responses.
Backend / API	Python with FastAPI	Asynchronous, high-performance request handling, input validation, and REST API layer between the frontend and the agent layer.
AI Orchestration	LangChain, LangGraph	Composable retrieval/tool chains (LangChain) and stateful, graph-structured multi-step agent workflows in the ReAct pattern (LangGraph).
LLM Providers	OpenAI API, Gemini API	Natural-language understanding and grounded response generation; dual-provider support allows cost/latency/quality trade-offs to be compared empirically.
Retrieval Paradigm	Retrieval-Augmented Generation (RAG)	Grounds every generated answer in retrieved institutional text, following Lewis et al. (2020), rather than relying on parametric LLM knowledge alone.
Embedding Model	Sentence-level dense encoder (Sentence-Transformers-class model)	Maps document chunks and queries into a shared semantic vector space for similarity search, following Reimers & Gurevych (2019).

Vector Database	ChromaDB, FAISS	Efficient storage and approximate nearest-neighbour similarity search over document embeddings (Johnson et al., 2019).
Evaluation Framework	RAGAs-derived metrics	Reference-free, component-level evaluation of retrieval and generation quality (Es et al., 2024).
Version Control	Git, GitHub	Source-code management, branching, and collaborative development across the team.

METHODOLOGY

The methodology describes the systematic approach followed to design, implement, and evaluate CampusGenie AI. The system follows a structured, layered methodology intended to maximize modularity, scalability, and ease of maintenance, integrating web application engineering with Retrieval-Augmented Generation and Agentic AI orchestration. The methodology proceeds in five stages: knowledge-base preparation, system architecture and implementation, the query-to-response flow, prompt and agent design, and empirical evaluation (the last of which is detailed separately in Section 10).

9.1 System Architecture

The system follows a six-layer architecture that separates presentation, request handling, agentic orchestration, retrieval, storage, and generation into independent, replaceable components. This separation of concerns improves testability (each layer can be evaluated in isolation), scalability (layers can be scaled or cached independently), and extensibility (a new capability can typically be added as a new tool at the agent layer without modifying the other layers).

Layer	Responsibility
Presentation Layer (React.js)	Captures the user's natural-language query, renders the conversation history, and displays CampusGenie AI's responses together with the source documents cited in generating them.
Application Layer (FastAPI)	Receives HTTP/WebSocket requests from the frontend, validates and sanitizes input, manages session/conversation state, and forwards queries to the agent layer.
Agent Layer (LangChain / LangGraph, ReAct pattern)	Reasons over the query, decides whether retrieval or another tool call is required, issues that call, observes the result, and repeats until a final answer can be produced — the interleaved thought-action-observation loop of Yao et al. (2023).
Retrieval Layer (RAG Engine)	Embeds the (possibly agent-rewritten) query, performs top-k approximate nearest-neighbour search against the vector index, and optionally re-ranks candidates before returning the most relevant document chunks.
Data Layer (ChromaDB / FAISS)	Stores dense vector embeddings of chunked institutional documents — notices, policies, timetables, placement circulars — together with source metadata for citation.
Generation Layer (LLM Provider)	Synthesizes a final, context-grounded natural-language answer conditioned on the retrieved chunks and the agent's reasoning trace, using the OpenAI or Gemini API.

9.2 Knowledge-Base Preparation (Data Pipeline)

Because RAG's grounding is only as good as the knowledge base it retrieves from, the data pipeline is treated as a first-class part of the methodology rather than a preliminary step.

- 1. Document collection: institutional notices, policy PDFs, timetables, and placement circulars are gathered from their existing sources without requiring any change to how the institution stores them.
- 2. Preprocessing and chunking: documents are cleaned (removing boilerplate/headers) and split into overlapping, semantically coherent chunks sized to balance retrieval precision against context completeness.
- 3. Embedding: each chunk is encoded into a dense vector using the selected sentence-level embedding model, mapping semantically similar text to nearby points in vector space.
- 4. Indexing: the resulting embeddings, together with source metadata (document name, section, last-updated date), are indexed in ChromaDB/FAISS for approximate nearest-neighbour search.
- 5. Incremental updates: when a source document changes, only the affected chunks are re-embedded and re-indexed, keeping the knowledge base current without a full rebuild.

9.3 Agent Design (ReAct Pattern)

The agent layer follows the ReAct reasoning-and-acting pattern (Yao et al., 2023), in which the LLM alternates between generating an explicit reasoning step ("thought"), selecting and invoking a tool such as the retrieval function ("action"), and incorporating the tool's result ("observation") before deciding whether to continue reasoning or produce a final answer. This is implemented as a stateful graph in LangGraph rather than a fixed linear chain, so that the agent can, for example, issue a second retrieval call to resolve a compound query ("what is my attendance percentage, and am I eligible for the mid-term examination?") by first retrieving the student's attendance record and then separately retrieving the eligibility policy, before combining both into a single synthesized answer. This design directly operationalizes objective O3 (Section 7) and is what distinguishes CampusGenie AI from single-pass RAG systems.

9.4 Query-to-Response Flow

- 1. The student or faculty member submits a natural-language query through the React.js conversational interface.
- 2. The FastAPI backend validates the request, attaches conversation context, and forwards it to the LangChain/LangGraph agent.
- 3. The agent reasons about the query ("thought") and decides whether retrieval — or a follow-up retrieval for a related sub-question — is required ("action").
- 4. The retrieval layer embeds the query, performs similarity search over the vector index, and returns the top-k most relevant document chunks together with their source metadata ("observation").
- 5. The agent evaluates whether the retrieved context is sufficient; if the query is compound, it may issue additional retrieval calls before proceeding.
- 6. The assembled context and reasoning trace are passed to the LLM (OpenAI or Gemini API), which generates a final, context-grounded natural-language answer.
- 7. The response, together with citations to the source documents used, is returned through the backend and rendered in the frontend for the user.

9.5 Prompt Engineering and Grounding Constraints

The generation prompt explicitly instructs the LLM to answer only from the retrieved context, to state clearly when the retrieved context is insufficient to answer confidently rather than fabricating a response, and to cite the specific source document(s) used — a design choice motivated directly by the hallucination taxonomy of Huang et al. (2023) and intended to make faithfulness (Section 10) a property the system is explicitly engineered for, not an incidental outcome.

9.6 Development Workflow

The backend is developed using Python and FastAPI, exposing versioned REST endpoints consumed by the React.js frontend; the agent, retrieval, and API-routing logic are organized into separate modules to keep the codebase maintainable and independently testable. Git and GitHub are used throughout for version control, feature branching, and collaborative review, providing the audit trail necessary to reproduce and iterate on system behaviour.

EVALUATION METHODOLOGY AND EXPECTED OUTCOMES

Consistent with objective O4 (Section 7) and the gap identified in Section 5.3, CampusGenie AI's effectiveness is assessed empirically rather than asserted. The evaluation methodology adapts the reference-free metric suite proposed by Es et al. (2024) in the RAGAs framework, applied to a held-out set of representative student and faculty queries spanning simple factual lookups, compound queries requiring multi-step reasoning, and out-of-scope queries the system should decline to answer rather than hallucinate.

10.1 Evaluation Metrics

Metric	Definition	What It Diagnoses
Context Precision	Proportion of retrieved chunks that are actually relevant to answering the query.	Measures retrieval quality — whether the retriever avoids surfacing irrelevant context.
Context Recall	Proportion of information required to answer the query that is present in the retrieved context.	Measures whether the retriever surfaces enough of the needed information.
Faithfulness	Proportion of claims in the generated answer that are directly supported by the retrieved context.	Directly measures hallucination — the central risk identified in Section 4.2.
Answer Relevance	Degree to which the generated answer actually addresses the question asked.	Measures generation quality independent of factual grounding.
End-to-End Latency	Wall-clock time from query submission to final rendered answer.	Measures usability — a factually perfect answer that arrives too slowly is still a design failure.

10.2 Evaluation Procedure

- Construct a representative evaluation query set, stratified into single-fact lookups, compound (multi-hop) queries, and deliberately unanswerable/out-of-scope queries.
- For each query, log the retrieved context, the agent's reasoning trace, and the final generated answer.
- Score each response against the metrics in Section 10.1, comparing (a) a naive single-pass RAG baseline against (b) the full agentic CampusGenie AI pipeline, to isolate the empirical contribution of the agentic reasoning layer.
- Measure end-to-end latency under representative concurrent load to validate the architecture's usability, not only its accuracy.
- Iterate on chunking strategy, retrieval top-k, and prompt design based on the diagnostic breakdown the metrics provide (e.g., low context recall motivates a larger top-k or better chunking; low faithfulness motivates stricter grounding instructions).

10.3 Expected Outcomes

The project is expected to produce: (i) a working, end-to-end CampusGenie AI system implementing the architecture described in Section 9; (ii) empirical evidence that the agentic, RAG-grounded design

achieves materially higher faithfulness and context recall than a naive single-pass RAG baseline on compound queries, consistent with the gap analysis in Section 5; (iii) a documented, reproducible evaluation methodology that can be reused as the knowledge base and query set grow; and (iv) a modular codebase demonstrably extensible with additional agent tools, validated by adding at least one illustrative extension (for example, a document-summarization tool) without modifying the core retrieval or presentation layers.

Software Architecture Diagram

MVP Architecture — RAG + LangGraph Agentic Pipeline

The diagram below illustrates the complete software architecture of CampusGenie AI, depicting the six-layer MVP design: Presentation (Student/Faculty interface), Application (FastAPI Backend), Agent (LangGraph ReAct Pattern with Thought–Action–Observation loop), Retrieval (Embed Query via Sentence-Transformers + Vector Search via ChromaDB/FAISS), Generation (LLM Generation Layer via OpenAI API / Gemini API), and Data Store (Policy Docs, Timetables, Circulars, Attendance).

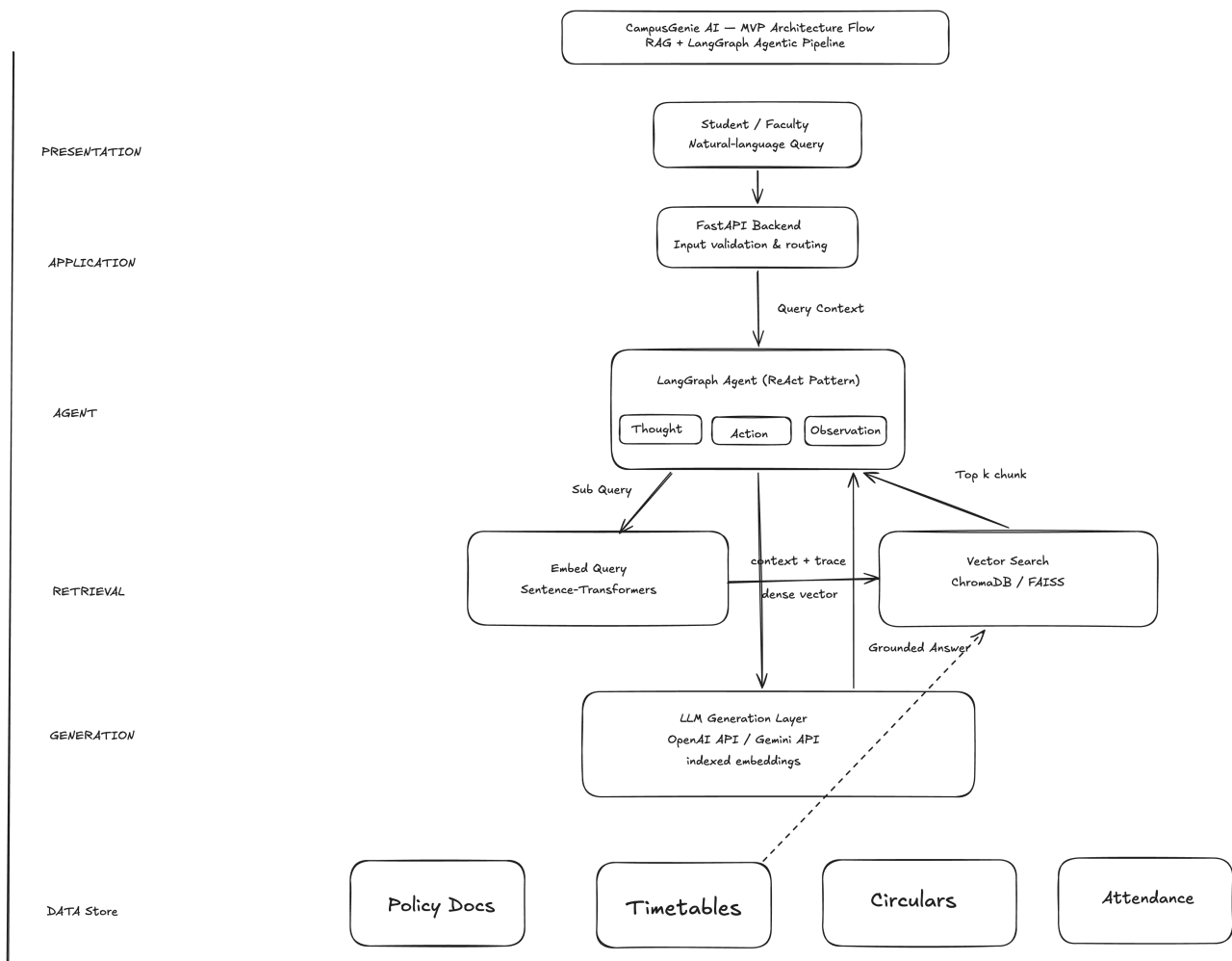
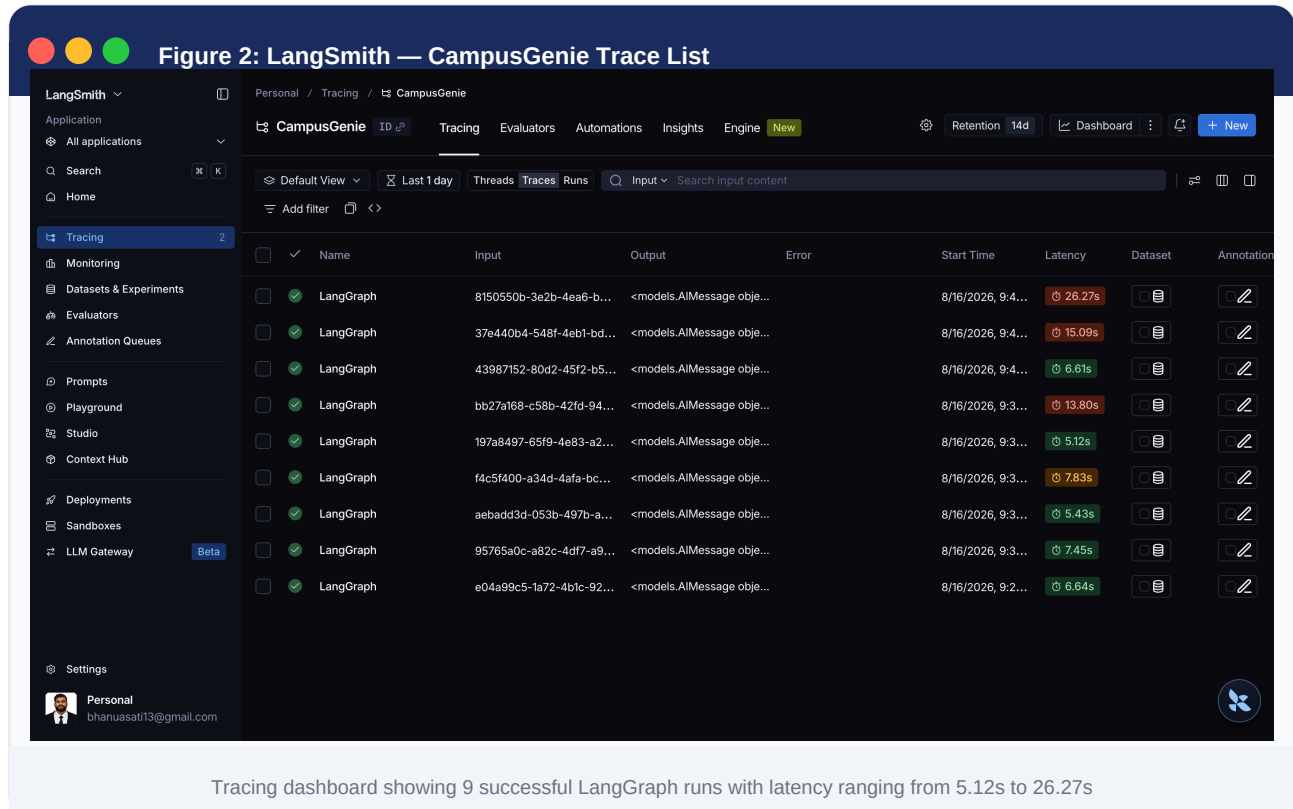


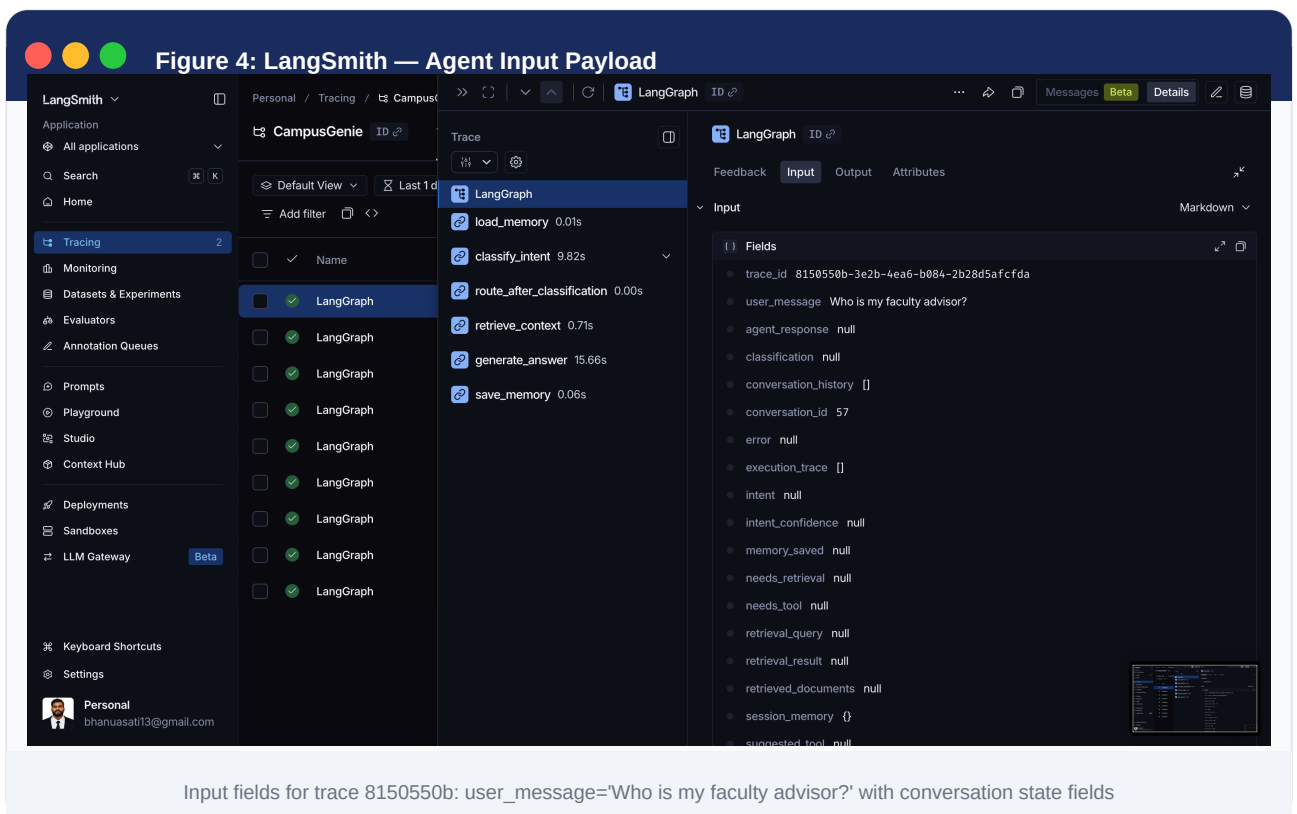
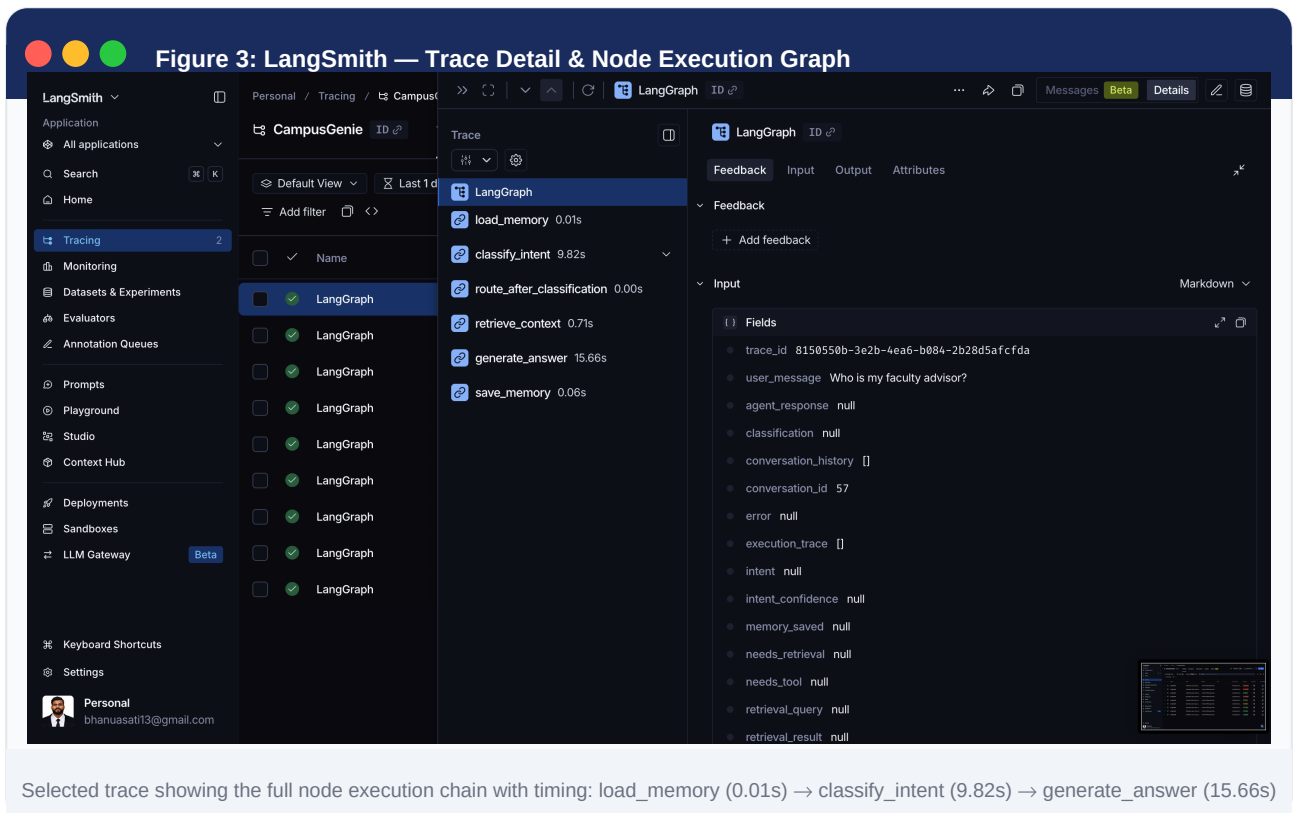
Figure 1: CampusGenie AI — MVP Software Architecture Flow (RAG + LangGraph Agentic Pipeline)

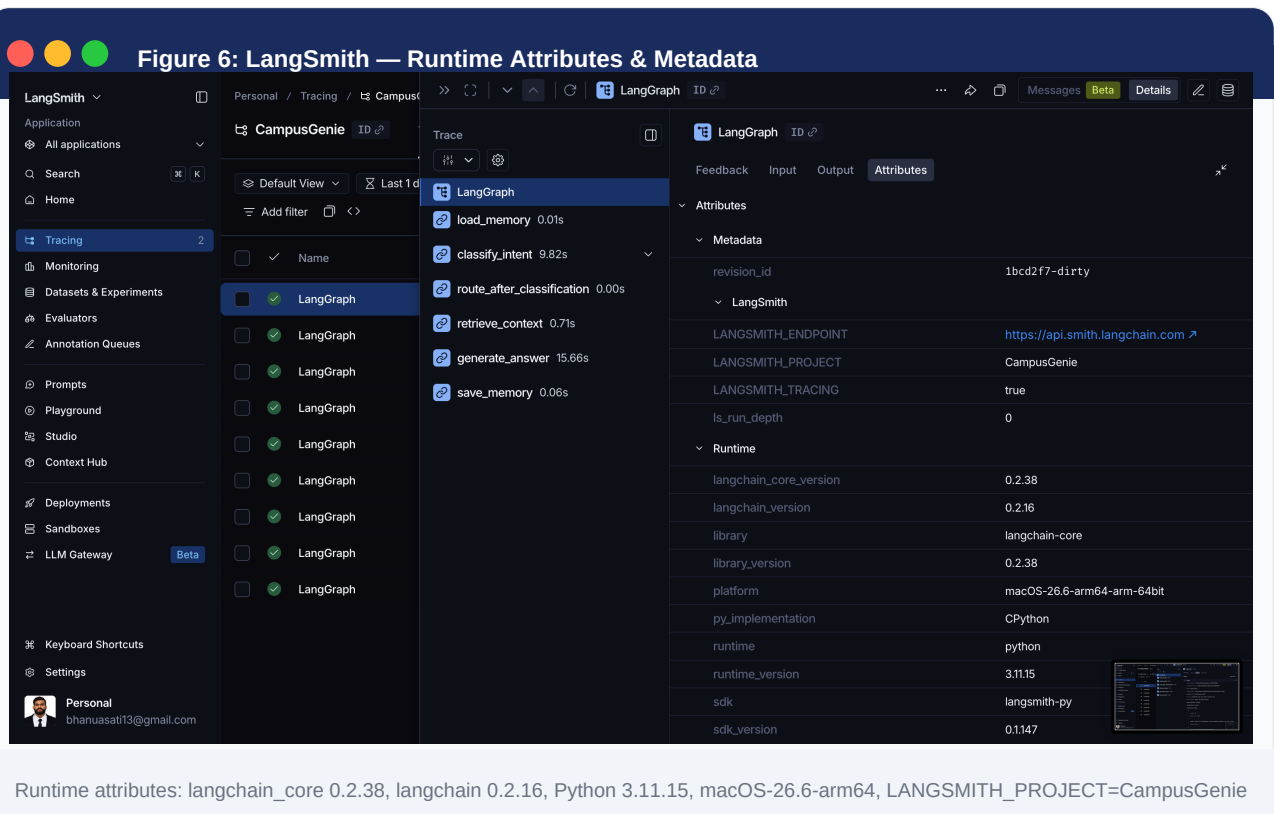
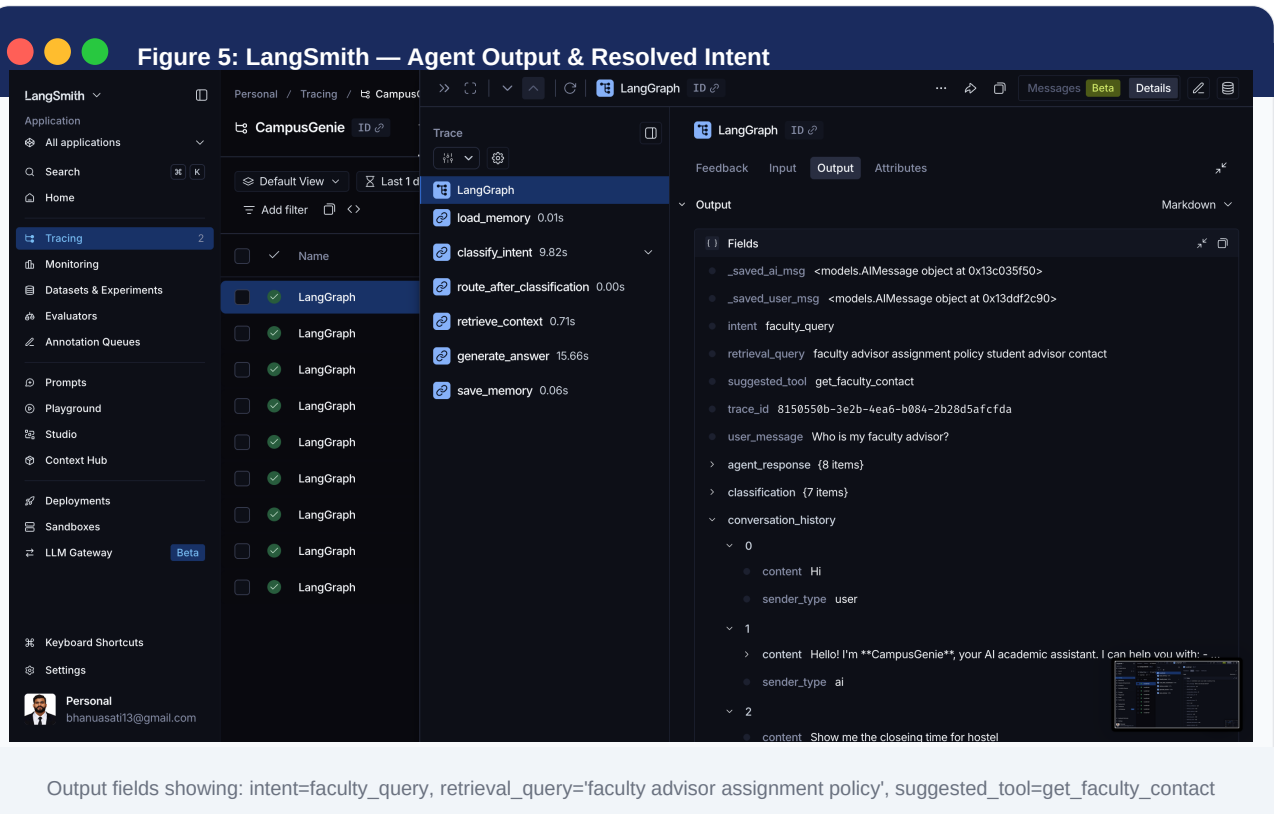
MVP Flow — System Tracing & Observability

LangSmith Tracing Dashboard — CampusGenie LangGraph Agent

The following screens demonstrate the real system traces captured in LangSmith during live execution of the CampusGenie AI LangGraph agent pipeline. Each trace shows the agent's node-level execution — `load_memory` → `classify_intent` → `route_after_classification` → `retrieve_context` → `generate_answer` → `save_memory` — along with inputs, outputs, latency, and runtime metadata.



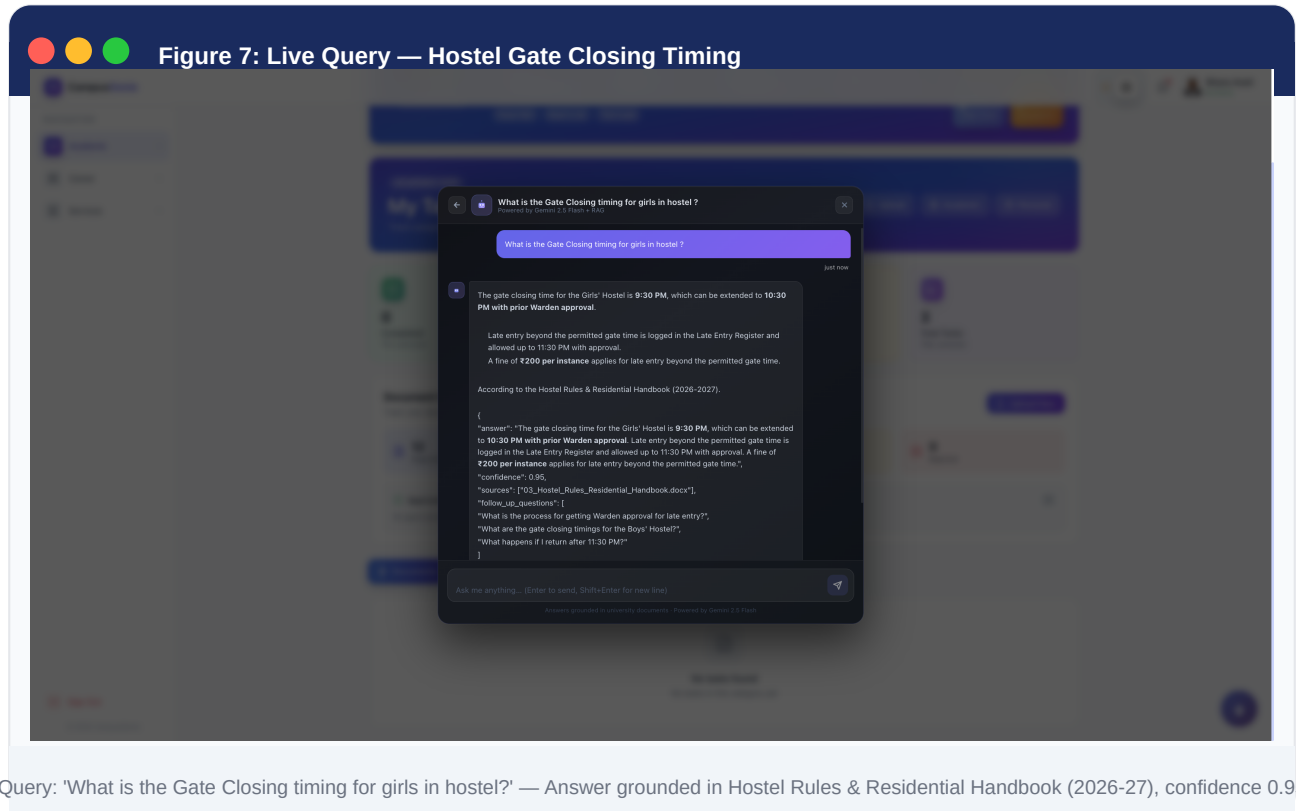


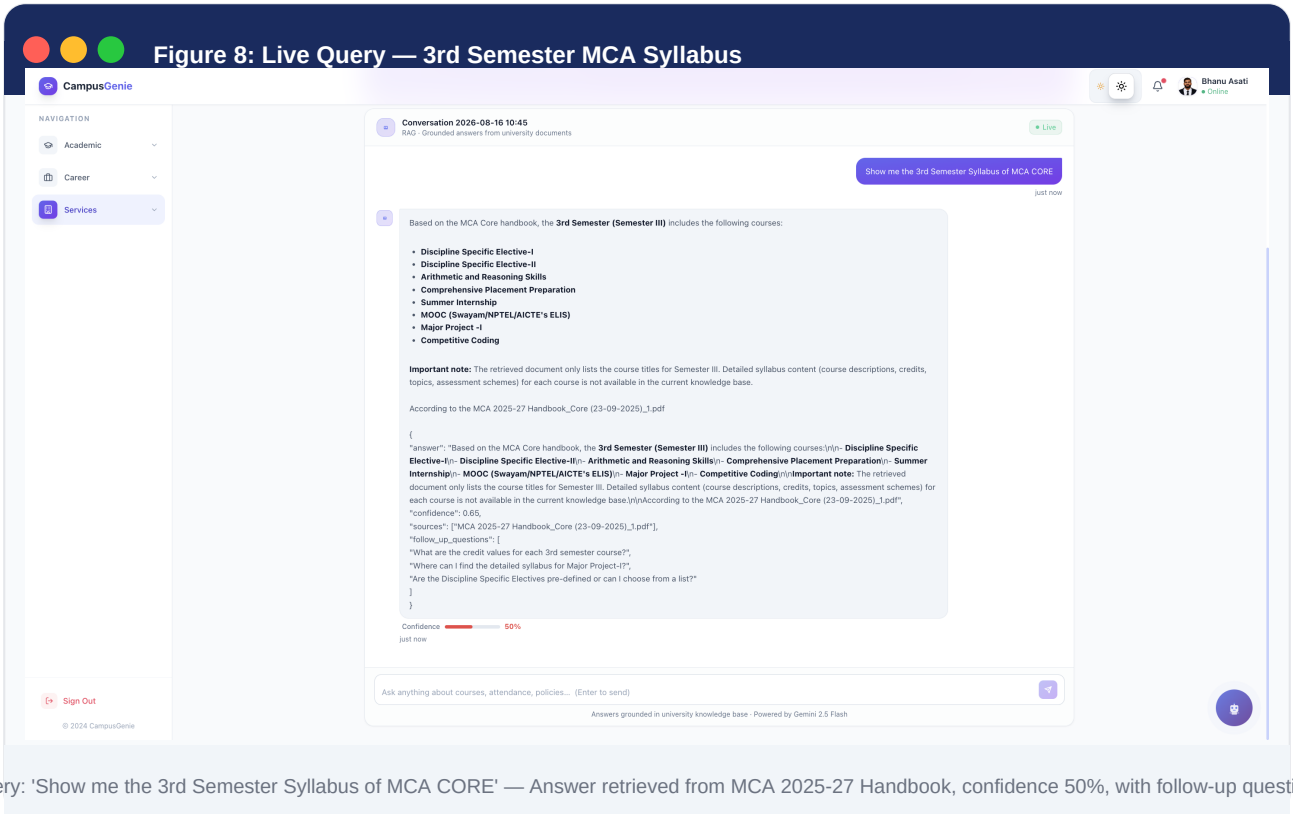


MVP Flow — Live Application Screens

CampusGenie AI Web Interface — Powered by Gemini 2.5 Flash + RAG

The screens below show the CampusGenie AI application running live, demonstrating grounded, citation-backed responses to real student queries. The system correctly retrieves answers from indexed university documents — hostel rules, course handbooks, and academic policies — and presents them with confidence scores and source references.



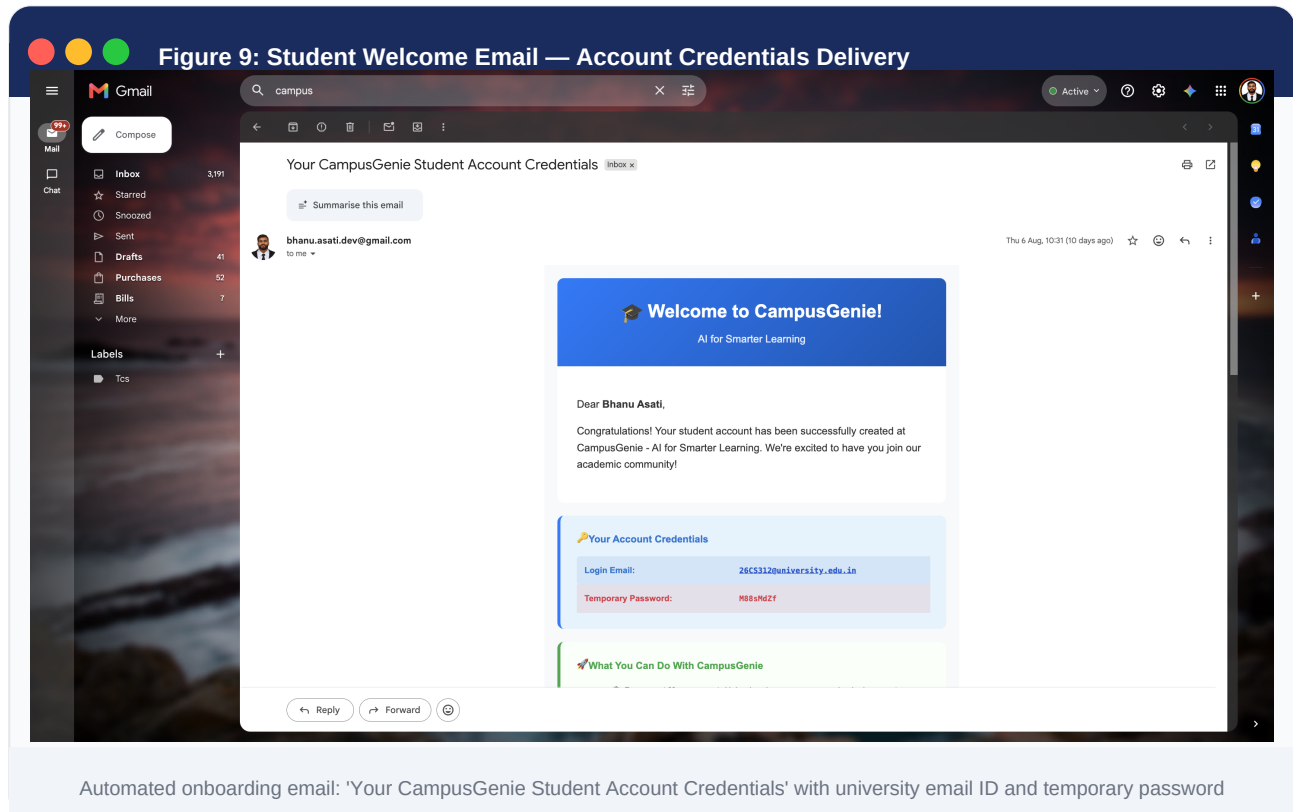


Query: 'Show me the 3rd Semester Syllabus of MCA CORE' — Answer retrieved from MCA 2025-27 Handbook, confidence 50%, with follow-up questions

MVP Flow — Student Onboarding & Email Notification

Automated Welcome Email with University Account Credentials

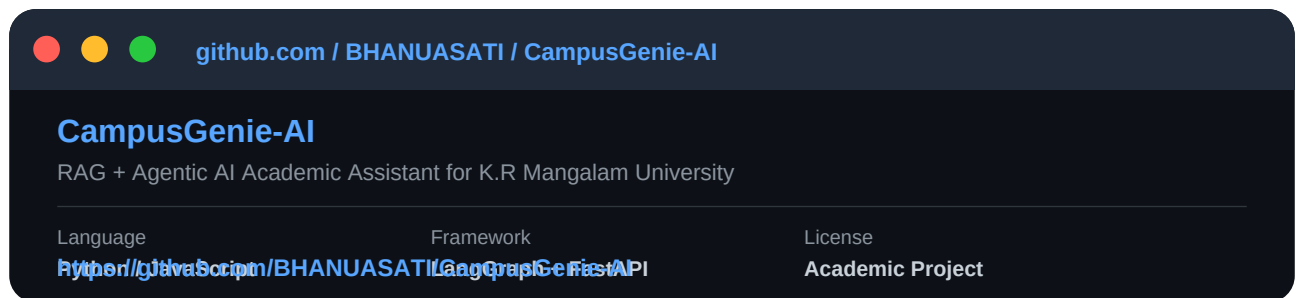
The screen below demonstrates the automated student onboarding flow: upon registration, CampusGenie sends a branded welcome email to the student's registered address containing their university login credentials (university email ID and a temporary password), enabling immediate access to the platform.



Repository Reference

Source Code, Architecture Diagrams & Documentation — GitHub

The complete source code, architecture diagrams, LangSmith trace exports, evaluation scripts, and iterative development history for CampusGenie AI are maintained in the GitHub repository listed below. The repository serves as the primary audit trail for all architectural decisions and knowledge-base preparation pipelines described in this synopsis.



Repository Structure

Directory / File	Contents
/frontend	React.js chat interface — query composer, response renderer, citation display
/backend	FastAPI application layer — routing, session management, API endpoints
/agent	LangGraph agent definition — ReAct graph, tool registry, prompt templates
/retrieval	RAG pipeline — document chunking, embedding, ChromaDB/FAISS indexing
/evaluation	RAGAs evaluation scripts — context precision, recall, faithfulness, answer relevance
/docs	Architecture diagrams, MVP flow screenshots, system design documentation
README.md	Project overview, setup instructions, environment configuration

LIMITATIONS AND FUTURE SCOPE

11.1 Limitations

- Retrieval quality is bounded by the completeness and currency of the underlying document corpus; information that has not yet been digitized or indexed cannot be retrieved, regardless of agent or model quality.
- LLM API dependence introduces external cost, rate-limit, and availability constraints outside the project's direct control, partially mitigated by supporting both OpenAI and Gemini as providers.
- Agentic, multi-step reasoning increases end-to-end latency relative to a single-pass RAG system, requiring a deliberate accuracy-latency trade-off that is measured, not assumed (Section 10).
- The current scope targets text-based institutional documents; multimodal sources (scanned handwritten notices, tabular timetable images) require additional preprocessing not covered in the initial implementation.

11.2 Future Scope

- Voice interaction: extending the presentation layer with speech-to-text and text-to-speech so queries can be spoken rather than typed.
- Document summarization: adding a summarization tool to the agent layer for long circulars and policy documents, reusing the existing retrieval infrastructure.
- Assignment assistance: an agent tool that helps structure or review academic submissions against institutional formatting or citation requirements.
- Personalized recommendations: extending the agent with a student-specific memory module (following the memory-module framing of Wang et al., 2023) to tailor academic and placement suggestions to an individual student's history.
- Mobile application: an Android client consuming the same FastAPI backend, extending access beyond the web interface.

REFERENCES

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
2. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
3. Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
4. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
5. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*.
6. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *Proceedings of the 11th International Conference on Learning Representations (ICLR)*.
7. Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2023). A Survey on Large Language Model based Autonomous Agents. *arXiv preprint arXiv:2308.11432*.
8. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997*.
9. Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2023). A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *arXiv preprint arXiv:2311.05232*.
10. Es, S., James, J., Espinosa Anke, L., & Schockaert, S. (2024). RAGAs: Automated Evaluation of Retrieval Augmented Generation. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL)*, 150–158.
11. Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing (3rd ed.)*. Pearson Education.
12. OpenAI. (2023). GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*.
13. Gemini Team, Google DeepMind. (2023/2024). Gemini: A Family of Highly Capable Multimodal Models. *arXiv preprint arXiv:2312.11805*.
14. LangChain AI. (2024). LangChain Documentation. Retrieved from <https://python.langchain.com>
15. LangChain AI. (2024). LangGraph Documentation. Retrieved from <https://langchain-ai.github.io/langgraph>
16. Chroma. (2024). ChromaDB Documentation. Retrieved from <https://www.trychroma.com>
17. Meta AI. (2024). FAISS: A Library for Efficient Similarity Search. Retrieved from <https://faiss.ai>

18. FastAPI. (2024). FastAPI Documentation. Retrieved from <https://fastapi.tiangolo.com>
19. React. (2024). React Documentation. Retrieved from <https://react.dev>
20. Asati, B., Dwivedi, S., et al. (2026). CampusGenie-AI [Source code repository]. GitHub. Retrieved from <https://github.com/BHANUASATI/CampusGenie-AI>